Highveld CO_2 Project

Project overview: what it is, why it exists, how it works, and what it does not show

Prepared for: Batsirai Mugomba
18 October 2026

What this document is

Document 1 of 2. It explains the project as a whole. Document 2 (the build tutorial) explains every file in the order it was built, so you can rebuild it.

Read this before showing the project to anyone

The pipeline has been verified on **synthetic** data (31 automated tests, a real PostGIS database, a clean-environment install).

It has **not yet been run on real OCO-2 data**: the machine that built it could not reach NASA's servers. Your first run on real files is the real test, and it may need small fixes.

Every number and figure in this document that comes from a run is **synthetic**, and says so. There are no scientific findings yet.

1. The project in one page

Question. Can a satellite that measures the total column of carbon dioxide see the small enhancements downwind of South Africa's coal power stations, and how often?

Approach. Take NASA's OCO-2 column-CO_2 (XCO_2) measurements over the South African Highveld, remove untrustworthy data, estimate for each satellite overpass what CO_2 would have been without nearby sources (the *background*), subtract it, find places where the remainder is significantly positive, and ask whether those places sit near known power plants more often than chance.

Item	Choice
Region	South African Highveld: 27–31°E, 24.5–28°S. The Mpumalanga coal-power cluster plus Johannesburg and Pretoria.
Satellite data	NASA OCO-2, Level 2 bias-corrected XCO_2 ("Lite"), product `OCO2_L2_Lite_FP`, daily NetCDF4 files
Source locations	WRI Global Power Plant Database v1.3.0 (real, CC BY 4.0): 16 fossil plants ≥ 100 MW in the box, plus Secunda (added by hand, approximate)
Language and tools	Python (pandas, NumPy, SciPy, scikit-learn, GeoPandas, netCDF4, Matplotlib, Folium), PostgreSQL/PostGIS, pytest, Jupyter
Outputs	Event tables, per-source detection frequencies, a control test, four figures, an interactive map, a PostGIS database
Size	About 2,000 lines of Python in 15 modules, 31 tests, one teaching notebook

2. Why this project exists

The purpose is to be a **portfolio repository** showing that you can take a remote-sensing problem from raw satellite files to an honest, reproducible result. It deliberately uses carbon dioxide instead of methane, to show that the skills transfer, while sharing the same architecture as the earlier methane exercise.

2.1 Skills it demonstrates

Skill	Where you can see it
Reading scientific data formats	`read_oco2.py`, `inspect_file.py`: NetCDF groups, fill values, masked arrays, time conversion
Geospatial analysis	`geo.py`, `sources.py`: projections, KD-tree distances, spatial clustering
Handling uncertainty	`anomaly.py`, `attribute.py`: robust statistics, standard errors, bootstrap over overpasses, Wilson intervals
Not fooling yourself	The control test, sensitivity analysis, calibrated null test, explicit limitations
Spatial databases	`schema.sql`, `db.py`: PostGIS geometry, GiST indexes, `ST_DWithin` on geography
Software engineering	Tests, configuration in one place, deterministic runs, one-command pipeline, clean-environment install
Communication	Figures designed for readability; documentation that states what was and was not verified

> **A note on how to present it**
>
> A hiring panel will trust a project that states its own limits more than one that overclaims. Say plainly what the data can and cannot show. Do not describe synthetic output as findings, and do not claim to have quantified emissions: this project detects and attributes enhancements, it does not estimate emission rates.

3. The science in plain terms

3.1 What the satellite measures

OCO-2 measures sunlight reflected from the ground through the whole atmosphere, in wavelength bands where CO_2 and oxygen absorb. From the absorption it infers **XCO_2**: the average CO_2 mole fraction in the whole column of air, in parts per million (ppm). Each retrieval is called a **sounding**.

OCO-2 scans a swath only about 10 km wide, as eight footprints side by side, with a new row of soundings roughly every 2 km along the track. It flies a ground-track pattern that repeats every 16 days.

3.2 Why CO_2 is harder than methane

Atmospheric CO_2 is about 415 ppm everywhere, and a large power-plant plume adds only about 1 to 2 ppm on top of that. Individual soundings are uncertain by roughly 0.5 to 1 ppm. The signal is therefore comparable to the noise, and you cannot find plumes by looking at XCO_2 itself.

Methane plume imagers have far greater contrast against their background. For CO_2 the realistic goal is statistical: combine many overpasses and ask whether enhancements occur near sources more often than elsewhere. Published work has shown that individual large power-plant plumes can be seen from OCO-2 under good conditions; if you cite this, read and cite the primary literature yourself.

3.3 The idea of a background and an anomaly

For each overpass the pipeline estimates the background: what XCO_2 would have been without the local sources. It uses only soundings farther than 40 km from any known source, taking the **median** in a running 100 km window along the track. The median is used because it is not dragged around by the odd plume. The **anomaly** is the measured XCO_2 minus this background.

Single soundings are too noisy to flag, so the pipeline averages them in 5 km along-track bins. A bin is flagged only if its mean anomaly is at least 1 ppm **and** at least 3 standard errors above zero. Neighbouring flagged bins merge into one **event**.

3.4 Why the sampling matters as much as the signal

A 10 km swath and a 16-day repeat mean that only about 2 to 3 overpasses in every 16 days cross the study box, and clouds remove many of those. Most power plants are seen on few days a year, and some are never seen at all. Because the pipeline keeps **every** sounding, not just detections, it can count how many passes actually observed each source and report *events divided by observations*. A bare count of detections cannot tell “rarely emits” from “rarely observed”.

4. How the pipeline works

Eight stages run in order. Each is its own module with its own tests.

#	Stage	Module	What it produces
1	Read	`read_oco2.py`	Soundings inside the box as a table

#	Stage	Module	What it produces
2	Quality control	`qc.py`	Only soundings flagged good, over land, nadir or glint mode
3	Sources	`sources.py`	Power plants, grouped into complexes (< 30 km apart), with a fast distance index
4	Anomalies	`anomaly.py`	Overpasses, along-track coordinate, background, anomaly, noise estimate
5	Events	`events.py`	5 km bins, flagged bins, merged events
6	Statistics	`attribute.py`, `composite.py`	Attribution, hotspots, detection frequency, control test, map grid, distance profile
7	Outputs	`mapping.py`, `pipeline.py`	CSV tables, `summary.json`, four figures, an interactive map
8	Database	`db.py`, `sql/schema.sql`	Everything loaded into PostGIS, ready for SQL

4.1 Three design decisions worth defending

- **One set of radii.** The background excludes soundings within 40 km of a source, so a plume further than 40 km downwind is partly absorbed into the background and cannot be detected. Attribution, the near zone of the control test, and the coverage radius are therefore all 40 km. Using a smaller attribution radius orphaned 29 % of events in testing.
- **A control group.** The pipeline compares the flagged rate near sources with the rate far from them (60 km or more). Flags far from any source are false alarms, so this measures how much noise the method mistakes for signal.
- **Bootstrap over overpasses, not bins.** Bins in one pass share a background and slowly varying errors, so they are not independent. Treating them as independent would give confidence intervals that are too narrow.

5. Architecture and data

5.1 Repository layout

Path	Role
`highveld_co2/`	The Python package (15 modules)
`tests/`	31 tests: unit, integration and a database round-trip
`notebooks/01_walkthrough.ipynb`	Guided walkthrough, executed once on synthetic data
`sql/schema.sql`	PostGIS tables and spatial indexes
`data/reference/`	Real power-plant locations (CSV)
`scripts/build_reference.py`	Rebuilds the power-plant file from the WRI database
`docker-compose.yml`, `Makefile`	One-command database and pipeline
`results/`	Empty until you run on real data; `results_demo/` holds synthetic output and is never committed

5.2 Database schema

Table	One row per	Geometry
`source_complexes`	group of nearby power plants	Point (WGS84)
`sources`	power plant or other source	Point
`passes`	satellite overpass of the box (valid or not)	none
`bins`	5 km along-track bin of a valid pass	Point
`events`	merged run of flagged bins	Point
`complex_stats`	source complex: passes observed, events, frequency, 95 % interval	none

5.3 Data sources and licences

- **OCO-2 L2 Lite XCO_2** (NASA GES DISC). Requires a free Earthdata account and authorising the "NASA GESDISC DATA ARCHIVE" application. Cite the DOI on the product page for the version you use.
- **Global Power Plant Database v1.3.0**, World Resources Institute, CC BY 4.0. A snapshot: Komati has since been retired and Kusile has expanded, so the list is out of date for recent years.
- **Secunda** (a coal-to-liquids plant, a very large CO_2 source that is not a power station) was added by hand with approximate coordinates. Verify them before relying on them.

6. What was verified, and what was not

Claim	Evidence	Status
Code runs end to end	Fresh virtual environment, `pip install -r requirements.txt`, tests and demo, on Python 3.13 with pandas 2.3 and 3.x	**Verified**
Analysis logic is correct	30 tests including known-answer checks (a planted plume becomes one event; pure noise gives almost none; the null test is calibrated)	**Verified**
Database step works	Loaded into PostGIS 3.4; NULLs stored correctly; SQL control test reproduces the Python counts exactly	**Verified**
Power-plant data are real	Downloaded from the WRI repository and filtered by script	**Verified**
Reads real OCO-2 files	Variable names follow the published Lite layout, but no real file was available to test against	**Not verified**
Downloader works	Helper functions are tested; the network calls to NASA could not be run	**Not verified**
Scientific results	None exist yet	**Not done**

> **Your first real-data run**
>
> Expect to spend your first session on step 5 of the README: run `inspect_file` on a real file and confirm the variable names. If one differs, you change one line in `read_oco2.py`.
>
> Then compare the measured scatter with the reported uncertainty, check the control test, and look at the coverage map **before** writing any conclusion.

7. What the synthetic demonstration shows

To test the pipeline, a generator writes OCO-2-format files with known answers: the real sampling pattern (three tracks, 16-day repeat), a background with correlated errors, cloud, plumes from the real plant locations blown by random winds, two urban CO_2 domes that are not power plants, and occasional retrieval artefacts that are not sources at all. Files are tagged synthetic and every figure is stamped. Two years of simulated data gave the following.

Quantity	Value
Overpass files / valid overpasses	137 / 135
Soundings kept by quality control	106,884 of 169,107 (63 %)
Bins / flagged bins / events	6,992 / 357 / 167
Flag rate near sources (≤ 40 km)	12.4 % (312 of 2526)
Flag rate far from sources (≥ 60 km): the false-alarm baseline	1.2 % (44 of 3792)
Rate ratio, with 95 % bootstrap interval	10.6 (7.0 to 16.6)
Measured scatter vs reported uncertainty	0.80 ppm vs 0.65 ppm

Do not expect these numbers on real data

The simulated plumes are strong and the simulated errors are mild. A real run will probably show a weaker contrast, a higher false-alarm rate, and more passes lost to cloud. What matters is that the *method* recovers a known answer, and that the control test would have exposed a failure.

7.1 Detection frequency per source complex (synthetic)

Complex	Passes observed	Passes with an event	Frequency (95 % interval)
Kendal power station complex	45	25	56 % (41–69 %)
Duvha power station complex	46	21	46 % (32–60 %)
Majuba power station	45	12	27 % (16–41 %)
Lethabo power station	44	12	27 % (16–42 %)
Tutuka power station	43	20	47 % (33–61 %)
Camden power station	0	0	never observed
Grootvlei power station	0	0	never observed
Kelvin power station	40	22	55 % (40–69 %)
Rooiwal power station complex	42	10	24 % (13–39 %)

Camden and Grootvlei were never observed: the simulated tracks pass too far away. Real tracks fall in different places, but the lesson is real: coverage gaps must be reported, not hidden.

7.2 Figures (all synthetic)

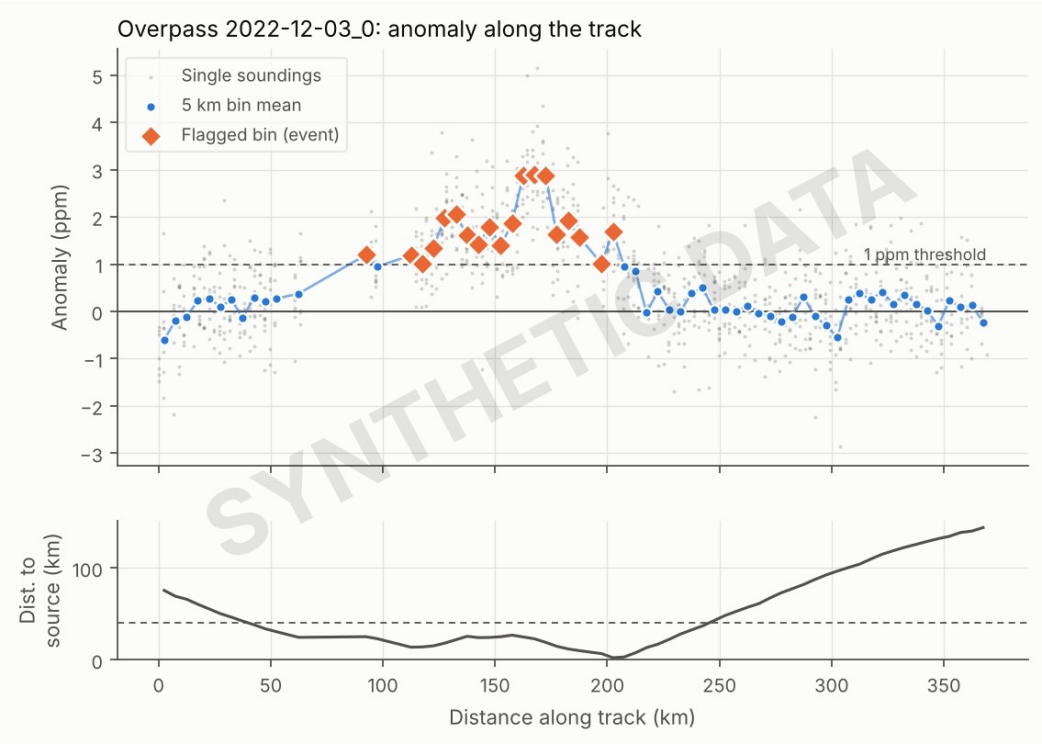


Figure 1. One synthetic overpass. Orange diamonds are flagged bins; the lower panel shows the track passing close to a source exactly where they occur.

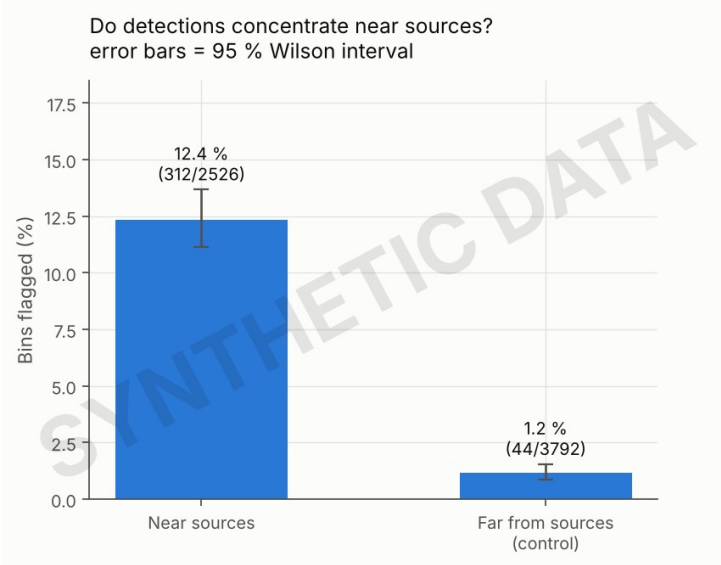


Figure 2. The control test: flagged-bin rate near sources against far from them.

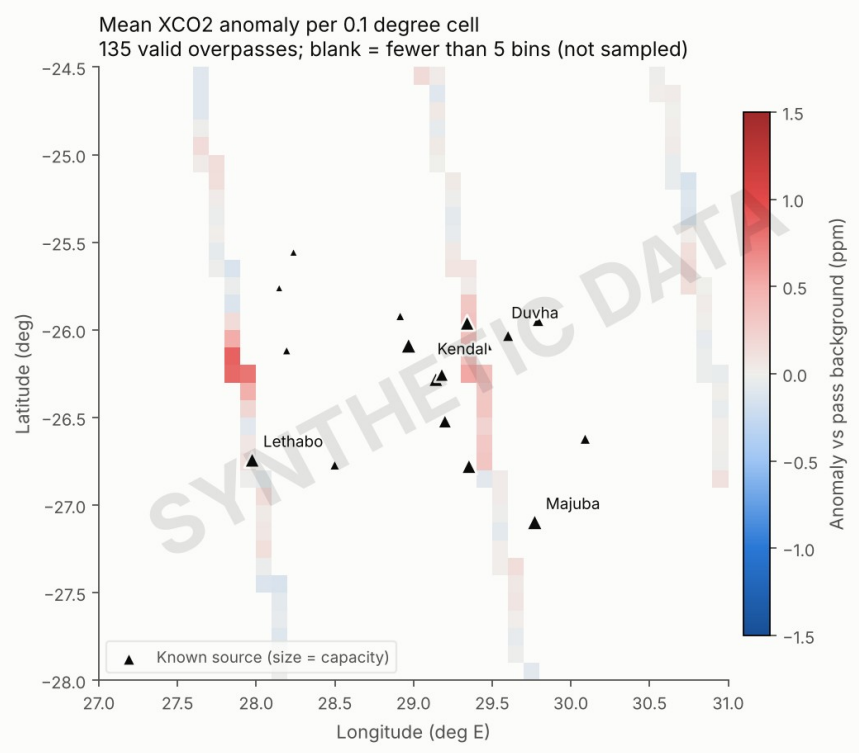


Figure 3. Mean anomaly per 0.1° cell. The blank space between the stripes is where the satellite never looked.

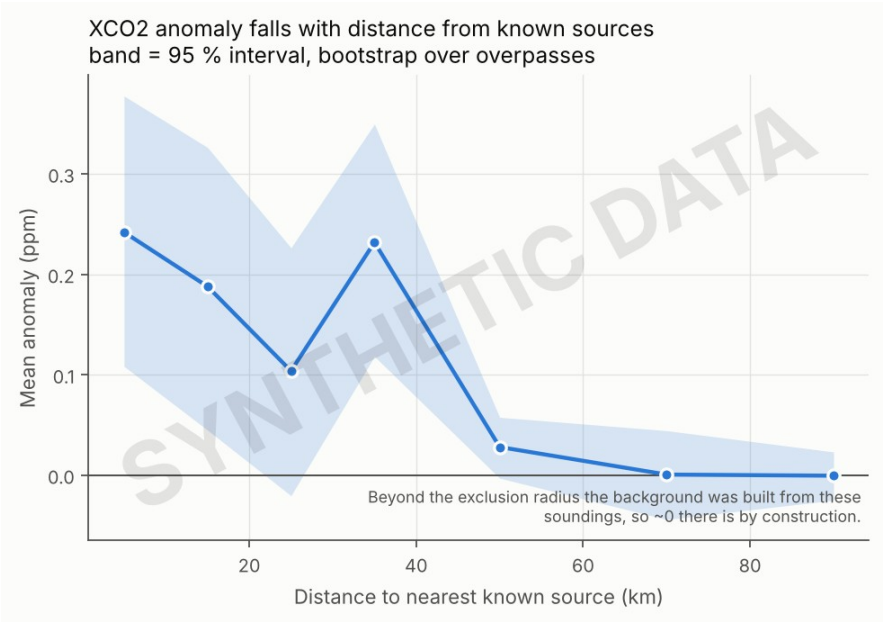


Figure 4. Mean anomaly against distance to the nearest source. Beyond the exclusion radius the profile is near zero by construction.

8. Limitations

Limitation	Consequence
Sparse sampling (10 km swath, 16-day repeat, clouds)	Most sources are seen on few days; some never. Detection frequency has wide intervals.
Small signal against noise	Plumes of 1–2 ppm sit close to the noise. Artefacts over bright or dark surfaces can look like plumes. The control test measures, but does not remove, this.

Limitation	Consequence
No wind data	Attribution is by proximity only. A plume can drift tens of kilometres, and the nearest source is not necessarily the upwind source.
Detection is not quantification	Seeing an enhancement near a plant does not give its emission rate. This project does not estimate emissions.
Urban CO_2	Johannesburg and Pretoria lie close to small power stations, so urban enhancements can be wrongly attributed to them. The synthetic data contains this trap deliberately.
Out-of-date source list	Plants have opened, closed or changed output since the database snapshot.
Complexes, not plants	Sources less than 30 km apart are merged. The instrument cannot separate them.
Background depends on a choice	The 40 km exclusion radius is a judgement. The notebook includes a sensitivity analysis; do the same on real data.

9. Where to take it next

1. **Run it on real data** and write the Results section of the README honestly.
2. **Add wind** (ERA5 or MERRA-2) and attribute events to the upwind source instead of the nearest one.
3. **Add a capacity check:** a 3 ppm event near a 180 MW plant is implausible.
4. **Use NO_2 as a tracer.** Coal plants co-emit NO_2, which Sentinel-5P sees at much higher contrast, so it can confirm candidate CO_2 events.
5. **Score against a published inventory** or other independent data to test attribution.
6. **Extend to a second region** (another coal belt or a megacity) by changing the box and the source list.

10. Glossary

Term	Meaning
XCO_2	Column-averaged dry-air CO_2 mole fraction, in ppm
ppm	Parts per million
Sounding	One retrieval of XCO_2 at one footprint
Overpass (pass)	One crossing of the study box by the satellite
Swath	The strip of ground the instrument sees: about 10 km wide for OCO-2
Background	Estimated XCO_2 without local sources, for one overpass
Anomaly	Measured XCO_2 minus background
MAD	Median absolute deviation; × 1.4826 gives a robust estimate of standard deviation
Standard error	Scatter divided by the square root of the number of soundings averaged
Event	A merged run of flagged along-track bins
Complex	Power plants closer than 30 km, chained together

Term	Meaning
Hotspot	Events closer than 20 km, grouped
Control test	Comparison of the flag rate near sources against far from them
Bootstrap	Resampling (here whole overpasses) to estimate how uncertain a statistic is
Wilson interval	A confidence interval for a proportion that behaves well for small counts
NetCDF	A self-describing scientific file format with groups and variables
PostGIS	The spatial extension of the PostgreSQL database
DBSCAN	A clustering method; with one minimum sample it chains neighbours within a set distance